Lab
2

BÁO CÁO BÀI THỰC HÀNH SỐ 2

Hồi quy Logistic và Hồi quy Softmax

Môn học: Học máy Thống kê
Lớp: DS102.R11.CNVN.1

Giảng viên hướng dẫn	Nguyễn Hiếu Nghĩa
Sinh viên thực hiện	24520003 - Nguyễn Thanh Hùng Anh

Phần 1. TỔNG QUAN DỮ LIỆU VÀ BÀI TOÁN

Ở bài lab trước, em đã thực hiện cào và dọn dẹp dữ liệu thẻ bài Yu-Gi-Oh! từ API của YGOPRODeck. Đến bài lab này, em dùng tập dữ liệu đã làm sạch đó để cài đặt mô hình phân loại.

1.1. Bài toán 1: Phân loại nhị phân Quái thú (Main Deck và Extra Deck)

- Dữ liệu: Em lọc riêng các lá bài là Quái thú (card_category == 0), bỏ hoàn toàn Spell và Trap.

- Đặc trưng đầu vào (X): Dùng 6 chỉ số chiến đấu gồm atk, def, level_rank, linkval, scale, link_marker_count. Ô nào bị rỗng thì em điền tạm bằng 0.

- Nhãn dự đoán (y): Cột is_extra_deck (1 là bài Extra Deck như Fusion, Synchro, Xyz, Link; 0 là bài Main Deck).

1.2. Bài toán 2: Phân loại đa nhãn các nhóm thẻ (Card Category)

- Dữ liệu: Dùng toàn bộ tập dữ liệu (cả Monster, Spell, Trap và Skill).

- Đặc trưng đầu vào (X): Gồm 6 chỉ số chiến đấu ở trên.

- Nhãn dự đoán (y): Cột card_category (0: Monster, 1: Spell, 2: Trap, 3: Skill, 4: Other).

Phần 2. ASSIGNMENT 1 (4 scores): LOGISTIC REGRESSION BẰNG NUMPY

2.1. Chia dữ liệu và tiền xử lý

- Phân tầng dữ liệu: Để tránh việc chia ngẫu nhiên làm tập Test bị thiếu bài Extra Deck, em tự viết hàm train_test_split bằng NumPy. Hàm này duyệt qua từng lớp, xáo trộn rồi cắt 80% làm Train, 20% làm Test nên tỷ lệ nhãn giữa 2 tập rất đều nhau.

- Chuẩn hóa: Em tính Mean và Std trên tập Train rồi trừ đi rồi chia cho Std để đưa dữ liệu về cùng một khoảng scale.

2.2. Kết quả chạy và Đồ thị Loss

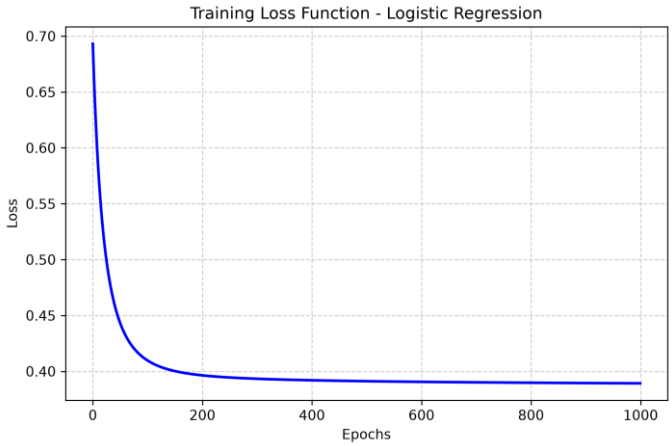
- Sau khi cho chạy 1000 epochs với learning_rate = 0.1, em thu được kết quả trên tập Test như sau:

+ Precision: 0.8269

+ Recall: 0.3909

+ F1 Score: 0.5309

- Đồ thị quá trình suy giảm Loss:



- Nhận xét: Nhìn vào biểu đồ em thấy đường loss giảm rất nhanh ở khoảng 200 epoch đầu, sau đó đi ngang và phẳng dần. Điều này cho thấy đoạn code Gradient Descent tự viết đã chạy đúng và mô hình đã hội tụ.

Phần 3. ASSIGNMENT 2 (4 scores): SOFTMAX REGRESSION BẰNG NUMPY

2.1 Chuyển đổi nhãn One-hot và Chia dữ liệu

- Em chuyển nhãn card_category thành mảng One-hot kích thước (N, 5) bằng np.eye(5)[y_indices].

- Em dùng lại hàm train_test_split tự viết ở bài 1 để chia phân tầng dựa trên nhãn nguyên ban đầu, sau đó mới cắt mảng One-hot theo index để tập Test không bị mất lớp.

2.2. Kết quả chạy và Đồ thị Loss

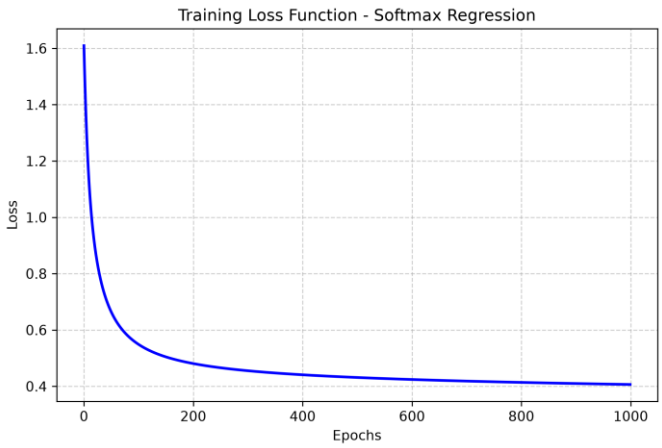
- Sau khi cho chạy 1000 epochs với learning_rate = 0.1, em thu được kết quả trên tập Test như sau:

+ Precision Macro: 0.3060

+ Recall Macro: 0.3946

+ F1 Score Macro: 0.3358

- Đồ thị quá trình suy giảm Loss:



- Nhận xét: Đồ thị Loss của bài đa lớp cũng giảm đều theo thời gian, chứng tỏ thuật toán Softmax cập nhật nhiều lớp cùng lúc bằng NumPy chạy ổn định.

Phần 4. ASSIGNMENT 3 (2 scores): THƯ VIỆN SCIKIT-LEARN

4.1. Logistic Regression

- Em dùng LogisticRegression của sklearn:

    + Precision Macro: 0.7300

    + Recall Macro: 0.3932

    + F1 Score Macro: 0.5111

4.2. Softmax Regression

- Em dùng LogisticRegression của sklearn, đầu vào y có kích thước (N,) do sklearn không cần đầu vào one-hot:

    + Precision Macro: 0.3109

    + Recall Macro: 0.3999

    + F1 Score Macro: 0.3426

4.3 Đánh giá

- Kết quả giữa code NumPy tự viết và thư viện sklearn là rất sát nhau. Thư viện sklearn dùng các thuật toán tối ưu nên chạy nhanh hơn, nhưng về chỉ số thì mô hình tự code bằng NumPy đã đáp ứng đúng lý thuyết.